

Université Hassan II de Casablanca
ENSET Mohammedia

Master SDIA (Smart Distributed and Intelligent Applications)
Parcours : SMA et IAD

PROJET DE FIN DE MODULE

Ecosystème Multi-Agent : Architecte Matériel PC

Réalisé par :

Marouane MAHBOUB

Abderahim FARAHI

Encadré par :

Pr. Sara RETAL

12 mai 2026

Table des matières

1	Introduction et Choix du Sujet	2
2	Détails d'Implémentation (Technologies Utilisées)	2
3	Architecture du Système Multi-Agent	2
3.1	Architecture Modulaire	2
3.2	Agents Spécialisés	3
3.2.1	Agent Architecte (ArchitectAgent)	3
3.2.2	Agent Procurement (ProcurementAgent)	3
3.2.3	Système Human-in-the-Loop (HumanLoopWorkflow)	3
3.3	Orchestration LangGraph	3
4	Défis Techniques et Résolution des Hallucinations	3
4.1	Hallucinations de Satisfaction d'Objectif	4
4.2	Limitations de la Recherche Sémantique	4
5	Evaluation et Optimisation des Prompts	4
5.1	Framework d'Evaluation Complet	4
5.1.1	Tests A/B Automatisés	4
5.1.2	Métriques de Performance	5
5.2	Résultats de l'Optimisation	5
5.2.1	Classement des Performances	5
5.2.2	Gains de Performance	5
6	Interface Utilisateur et Human-in-the-Loop	5
6.1	Architecture Modulaire du Human-in-the-Loop	5
6.1.1	Gestionnaire d'Approbation (ApprovalManager)	5
6.1.2	Interface Utilisateur Modulaire (HumanApprovalUI)	6
6.1.3	Intégration Workflow (HumanLoopWorkflow)	6
6.2	Implémentation Technique	6
6.3	Fonctionnalités Avancées	6
6.3.1	Validation Automatique de Sécurité	6
6.3.2	Persistance et Audit	6
7	Structure du Projet	7
8	Conclusion et Perspectives	7
8.1	Réalisations Accomplies	7
8.2	Innovations Techniques	7
8.3	Impact Académique	8
8.4	Perspectives d'Amélioration	8
8.5	Validation Expérimentale	8

1 Introduction et Choix du Sujet

Pour ce projet de fin de module, nous avons choisi de développer un **Architecte Matériel PC (Custom PC Builder)**. Le choix de ce sujet n'est pas fortuit : l'assemblage d'un ordinateur est un problème complexe d'ingénierie et de satisfaction de contraintes.

Pourquoi ce sujet ?

- **Pertinence pour le Multi-Agent** : Contrairement à un chatbot classique, construire un PC nécessite plusieurs expertises distinctes : planifier un budget, vérifier la compatibilité physique (Sockets, DDR4 vs DDR5), et acheter au meilleur prix.
- **Nécessité du RAG** : Le marché du hardware évolue chaque jour. Un grand modèle de langage (LLM) seul proposerait des composants obsolètes ou hors stock. L'utilisation du RAG était indispensable pour connecter l'agent à un inventaire local en temps réel.
- **Besoin du Human-in-the-Loop** : L'achat de matériel coûtant parfois des dizaines de milliers de dirhams, il est impensable de laisser l'IA finaliser une transaction sans l'approbation humaine finale.

2 Détails d'Implémentation (Technologies Utilisées)

Le code source de notre projet a été structuré de manière modulaire autour des technologies de pointe de l'écosystème IA :

- **LangGraph** : Utilisé pour concevoir l'orchestrateur (StateGraph). Il maintient l'état global (BuilderState) et permet d'interrompre l'exécution via `interrupt_before`.
- **ChromaDB et LangChain** : Utilisés pour la base de données vectorielle (RAG). Nous avons vectorisé nos propres bases de données CSV (inventaire de CPUs, GPUs, etc.) en utilisant des embeddings locaux.
- **Pandas** : Implémenté comme mécanisme de sécurité (Fallback). Si l'API LLM tombe en panne à cause des limites de requêtes (Rate Limits), un script Pandas déterministe prend le relais pour garantir la création du devis sans interruption.
- **Flask / SocketIO** : Déploiement de l'interface utilisateur (UI) permettant le chat en temps réel et l'apparition dynamique des boutons de validation humaine.

3 Architecture du Système Multi-Agent

Dans le cadre de ce projet, nous avons développé un écosystème intelligent capable de concevoir un ordinateur optimisé selon un budget précis. Le flux de travail a été décomposé hiérarchiquement avec une architecture modulaire avancée.

3.1 Architecture Modulaire

Le système a été structuré en modules indépendants pour améliorer la lisibilité, la maintenabilité et la réutilisabilité :

- **Module agents/nodes.py** : Contient les implémentations des agents individuels avec des classes spécialisées (ArchitectAgent, ProcurementAgent)
- **Module human_loop/** : Système complet de validation humaine avec gestion d'état, interface utilisateur et intégration workflow

- **Module evaluation/** : Framework d'évaluation et d'optimisation des prompts avec tests A/B
- **Module rag/** : Système de récupération augmentée avec base vectorielle ChromaDB

3.2 Agents Spécialisés

3.2.1 Agent Architecte (**ArchitectAgent**)

Responsabilités principales :

- Analyse sémantique des requêtes utilisateur
- Extraction intelligente du budget avec patterns regex multiples
- Classification automatique des cas d'usage (gaming, IA/ML, bureautique, création de contenu)
- Détermination du niveau de performance requis

3.2.2 Agent Procurement (**ProcurementAgent**)

Responsabilités principales :

- Interrogation de la base RAG avec filtrage catégoriel
- Algorithme d'optimisation avec vérification de compatibilité matérielle
- Génération de devis formatés avec métriques de performance
- Validation automatique des contraintes budgétaires

3.2.3 Système Human-in-the-Loop (**HumanLoopWorkflow**)

Responsabilités principales :

- Gestionnaire d'approbation avec états persistants
- Interface utilisateur modulaire avec composants Flask et SocketIO
- Intégration transparente avec LangGraph
- Historique et statistiques des décisions humaines

3.3 Orchestration LangGraph

L'orchestrateur utilise un StateGraph avec état typé (**BuilderState**) étendu pour supporter :

- Gestion des métadonnées d'approbation (`approval_request_id`, `approval_status`)
- Suivi des analyses de requirements (`requirements_analysis`)
- Persistance des composants sélectionnés (`selected_components`)
- Intégration avec le système de validation humaine

4 Défis Techniques et Résolution des Hallucinations

Le développement de ce système nous a confrontés à plusieurs défis d'ingénierie inhérents aux LLMs.

4.1 Hallucinations de Satisfaction d'Objectif

Le Problème : Initialement, l'Agent Procurement mentait sur les prix ou inventait des noms de composants inexistantes (ex : "NVIDIA Radeon") dans le but mathématique de satisfaire parfaitement le budget imposé par l'utilisateur.

La Solution : Mise en place d'un mécanisme de Chain-of-Thought (CoT) via une balise <thinking>. En forçant l'agent à extraire les vrais prix du contexte RAG et à écrire ses calculs d'addition étape par étape avant de générer le devis final, nous avons drastiquement réduit les erreurs logiques et budgétaires.

4.2 Limitations de la Recherche Sémantique

Le Problème : La base vectorielle ChromaDB filtrait parfois des composants critiques. La similitude sémantique échouait à comprendre qu'une requête pour du "High-End" correspondait concrètement à un "Core i7" ou un "Ryzen 9".

La Solution : Passage à une Recherche Catégorielle Structurée. Au lieu d'une seule recherche globale, le système itère sur un tableau de 8 catégories (CPUs, GPUs, RAM, etc.) et effectue une récupération spécifique pour chacune.

Listing 1 – Recherche catégorielle RAG

```
categories = ["CPUS", "GPUS", "MOTHERBOARDS", "RAM",  
             "STORAGE", "COOLERS", "PSUS", "CASES"]  
full_context = []  
  
for cat in categories:  
    results = db.as_retriever(  
        search_kwargs={"k": 2, "filter": {"category": cat}}  
    ).invoke(plan)  
    for doc in results:  
        full_context.append(f"[CATEGORY: {cat}] {doc.page_content}")
```

5 Evaluation et Optimisation des Prompts

5.1 Framework d'Evaluation Complet

Nous avons développé un système d'évaluation sophistiqué pour optimiser les performances des prompts utilisés par nos agents.

5.1.1 Tests A/B Automatisés

Le framework implémente une méthodologie de tests A/B rigoureuse :

- **4 Stratégies de Prompts :** Basic, Detailed, Structured, Context-Aware
- **6 Scénarios de Test :** Gaming, Workstation, Office, AI/ML, Streaming, Budget constraint
- **Métriques Multiples :** Précision, adhérence budgétaire, compatibilité, temps de réponse, satisfaction utilisateur
- **Analyse Statistique :** Moyenne, médiane, écart-type, intervalles de confiance

5.1.2 Métriques de Performance

Le système évalue cinq dimensions critiques :

- **Précision d'Extraction** : Capacité à extraire correctement le budget et les exigences
- **Adhérence Budgétaire** : Respect des contraintes financières (tolérance plus ou moins 10 pourcent)
- **Compatibilité Matérielle** : Vérification des compatibilités CPU-Motherboard, RAM-CPU
- **Temps de Réponse** : Performance temporelle des différentes stratégies
- **Satisfaction Utilisateur** : Score composite basé sur les autres métriques

5.2 Résultats de l'Optimisation

Les tests A/B ont révélé des différences significatives entre les stratégies :

5.2.1 Classement des Performances

1. **Strategy D (Context-Aware)** : Score 0.765 - Meilleure performance globale
2. **Strategy C (Structured)** : Score 0.764 - Excellente précision (0.800)
3. **Strategy B (Detailed)** : Score 0.674 - Bon compromis performance/temps
4. **Strategy A (Basic)** : Score 0.542 - Baseline de référence

5.2.2 Gains de Performance

- **Amélioration maximale** : 41 pourcent entre la meilleure et la pire stratégie
- **Temps de réponse optimal** : Strategy A (1.75s) vs Strategy C (1.80s)
- **Précision maximale** : Strategy C avec 80 pourcent de précision d'extraction
- **Robustesse** : Strategy D excellente sur tous les scénarios de complexité

6 Interface Utilisateur et Human-in-the-Loop

6.1 Architecture Modulaire du Human-in-the-Loop

Le système de validation humaine a été entièrement refactorisé en une architecture modulaire comprenant trois composants principaux.

6.1.1 Gestionnaire d'Approbation (ApprovalManager)

Classe centrale gérant le cycle de vie des demandes d'approbation :

- **Création de requêtes** : Génération d'identifiants uniques et horodatage
- **Validation de sécurité** : Vérification automatique des configurations
- **Traitement des décisions** : Gestion des approbations/rejets avec feedback
- **Historique persistant** : Sauvegarde et export des décisions pour audit

6.1.2 Interface Utilisateur Modulaire (HumanApprovalUI)

Composants Flask/SocketIO spécialisés pour l'interaction humaine :

- Interface d'approbation avec boutons contextuels
- Comparaison requête/build côte-à-côte
- Collecte de feedback avec zone de texte
- Statistiques en temps réel et historique

6.1.3 Intégration Workflow (HumanLoopWorkflow)

Pont entre le système d'approbation et LangGraph :

- Génération automatique de nœuds LangGraph
- Synchronisation entre l'état workflow et les requêtes
- Nettoyage automatique des requêtes expirées
- Sauvegarde et restauration de l'état workflow

6.2 Implémentation Technique

Listing 2 – Intégration modulaire du Human-in-the-Loop

```
# Initialisation du systeme modulaire
from human_loop import (
    approval_manager,
    create_approval_ui,
    initialize_workflow_integration
)

# Configuration de l'integration workflow
workflow_integration = initialize_workflow_integration(
    approval_manager
)
approval_ui = create_approval_ui(approval_manager)

# Creation du noeud d'approbation pour LangGraph
human_approval_node = workflow_integration.create_human_approval_node()

# Integration dans le workflow
workflow.add_node("human_approval", human_approval_node)
```

6.3 Fonctionnalités Avancées

6.3.1 Validation Automatique de Sécurité

Le système effectue des vérifications automatiques :

- Validation budgétaire avec tolérance de 10 pourcent
- Contrôle de cohérence des composants essentiels
- Vérification de format du devis généré
- Détection d'anomalies dans les prix

6.3.2 Persistance et Audit

- Export JSON avec sauvegarde automatique des décisions

- Statistiques d'approbation (taux, temps de décision moyen)
- Traçabilité complète avec historique et feedback
- Nettoyage automatique des requêtes expirées

7 Structure du Projet

Le projet est organisé de manière modulaire :

Dossier agents/ : Système multi-agent

- graph.py : Orchestration LangGraph
- nodes.py : Implémentations des agents

Dossier human_loop/ : Système Human-in-the-Loop

- approval_manager.py : Gestionnaire d'approbation
- ui_components.py : Interface utilisateur
- workflow_integration.py : Intégration workflow

Dossier evaluation/ : Framework d'évaluation

- prompt_evaluation.py : Tests de performance
- ab_testing.py : Tests A/B
- metrics_dashboard.py : Dashboard de monitoring

Dossier rag/ : Système RAG

- ingest.py : Ingestion des données
- chroma_db/ : Base vectorielle

Dossier data/ : Bases de données composants (CSV)

Fichier app.py : Interface principale Flask

8 Conclusion et Perspectives

8.1 Réalisations Accomplies

Ce projet démontre la puissance d'une architecture multi-agent modulaire et orchestrée. Nous avons réussi à implémenter tous les composants requis :

- **Orchestration Multi-Agent** : Architecture LangGraph avec agents spécialisés et modulaires
- **RAG Agentique** : Système de récupération augmentée avec base vectorielle ChromaDB
- **Human-in-the-Loop Avancé** : Système complet de validation avec interface modulaire
- **Interface Web Fonctionnelle** : Application Flask avec chat temps réel
- **Evaluation des Prompts** : Framework complet avec tests A/B et optimisation

8.2 Innovations Techniques

- **Architecture Modulaire** : Séparation claire des responsabilités
- **Système d'Approbation Sophistiqué** : Gestion d'état persistant et validation automatique
- **Optimisation Basée sur les Données** : Tests A/B avec amélioration de 41 pourcent
- **Robustesse Opérationnelle** : Mécanismes de fallback et gestion d'erreurs

8.3 Impact Académique

Le système illustre l'application pratique des concepts théoriques du cours SMA et IAD :

- Coordination Multi-Agent avec orchestration séquentielle
- Interaction Homme-Machine avec validation humaine
- Optimisation Algorithmique par évaluation empirique
- Ingénierie Logicielle avec architecture maintenable

8.4 Perspectives d'Amélioration

- Agent Vérificateur Indépendant pour simulation de benchmarks
- Optimisation Multi-Objectifs avec algorithmes génétiques
- Apprentissage Adaptatif basé sur l'historique des approbations
- Intégration E-commerce avec APIs de fournisseurs
- Système de Recommandation avec ML pour suggestions personnalisées

8.5 Validation Expérimentale

Les tests A/B ont démontré l'efficacité de notre approche avec des améliorations mesurables de 41 pourcent entre les stratégies de prompts, validant scientifiquement notre méthodologie d'optimisation.

Lien vers le code source (GitHub) :

<https://github.com/votre-lien-ici>